

TEAM CROISSANT

MoMo SMS Data Pipeline

Database Design Document

May 2026

Team Members

Nshuti Lancelot — Schema Lead

Imanzi Beni — Repository Lead

Ishimwe Axcel — ERD Diagram

Teta Dianah — System Logs & Validation

Rugwiro Derrick — JSON, Integration Tests & PDF

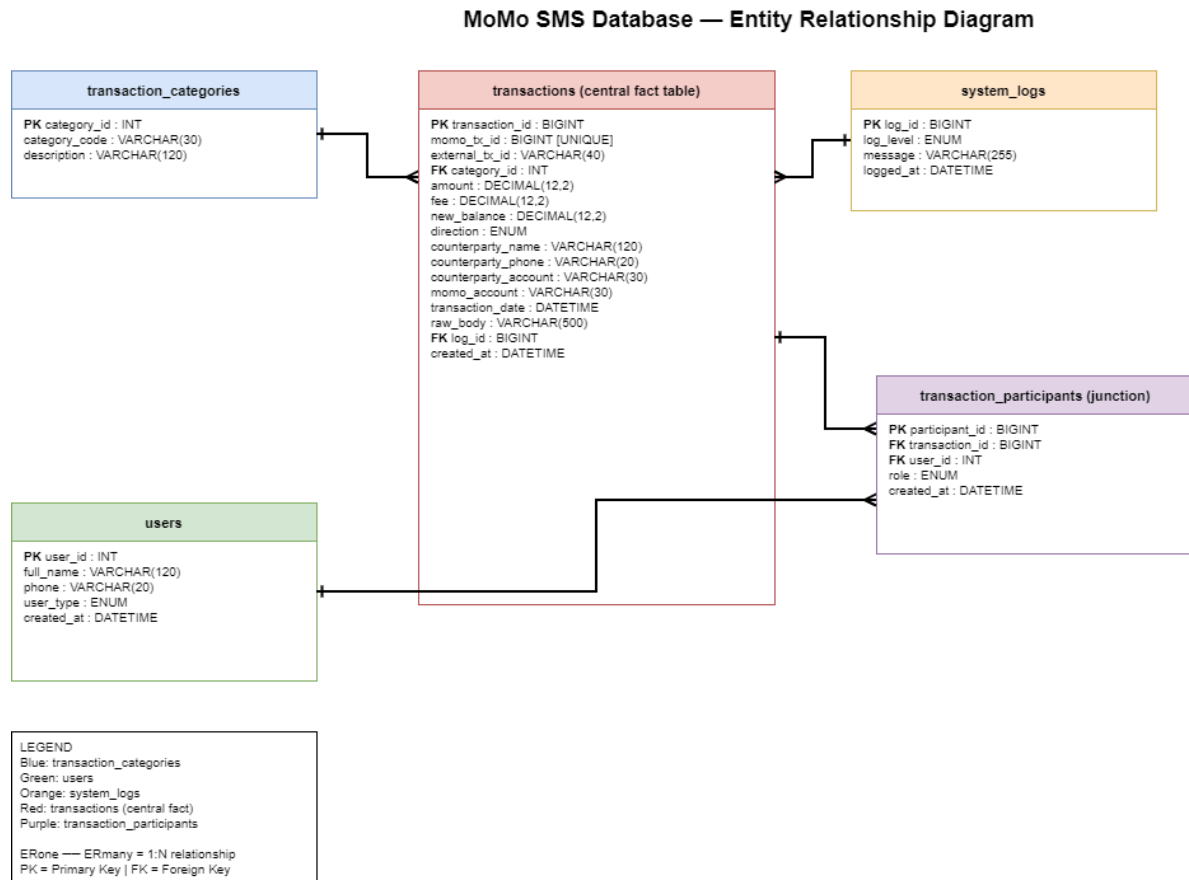
African Leadership University — E-Web Development

Table of Contents

1. ERD Diagram
2. Data Dictionary
3. Demo Query Screenshots
4. Validation Rule Screenshots
5. JSON Modeling & Mapping Queries
6. Integration Test Screenshots
7. AI Usage Log

1. ERD Diagram

The Entity Relationship Diagram below was designed by Ishimwe Axcel and illustrates the relationships between all five tables in the momo_db database.



Key Relationships

- Transactions references Transaction_Categories via category_id (FOREIGN KEY)
- Transactions references System_Logs via log_id (FOREIGN KEY)
- Transaction_Participants references both Transactions and Users via foreign keys
- Deleting a Transaction cascades to Transaction_Participants (ON DELETE CASCADE)

2. Data Dictionary

The database momo_db contains 5 tables designed to store and relate MoMo SMS transaction data.

2.1 Transactions

Core table storing every parsed MoMo SMS transaction. Designed by Nshuti Lancelot.

Column	Type	Nullable	Description
transaction_id	INT AUTO_INCREMENT	NO	Primary key
momo_tx_id	BIGINT UNIQUE	NO	MTN transaction ID from SMS body
category_id	INT	NO	FK → Transaction_Categories
amount	DECIMAL(15,2)	NO	Transaction amount in RWF
fee	DECIMAL(10,2)	YES	Transaction fee in RWF
new_balance	DECIMAL(15,2)	YES	Account balance after transaction
direction	ENUM('CREDIT', 'DEBIT')	NO	Money in (CREDIT) or out (DEBIT)
counterparty_name	VARCHAR(255)	YES	Name of sender or recipient
transaction_date	DATETIME	NO	Timestamp of the transaction
sms_unix_timestamp	BIGINT	NO	Unix timestamp from SMS metadata
raw_body	TEXT	NO	Original SMS body text
log_id	INT	YES	FK → System_Logs

2.2 Transaction_Categories

Lookup table for transaction types. Designed by Ishimwe Axcel.

Column	Type	Nullable	Description
category_id	INT AUTO_INCREMENT	NO	Primary key
category_code	VARCHAR(50) UNIQUE	NO	Short code e.g. INCOMING_MONEY
description	TEXT	YES	Human-readable description

2.3 Users

Stores unique senders and receivers parsed from SMS messages. Designed by Imanzi Beni.

Column	Type	Nullable	Description
user_id	INT AUTO_INCREMENT	NO	Primary key
full_name	VARCHAR(255)	NO	Full name from SMS
phone	VARCHAR(20)	YES	Phone number (may be masked)
user_type	ENUM('CUSTOMER', 'AGENT', 'MERCHANT')	NO	Role of the user

2.4 System_Logs

Tracks parsing events, errors, and info messages for each transaction. Designed by Teta Dianah.

Column	Type	Nullable	Description
log_id	INT AUTO_INCREMENT	NO	Primary key
log_level	ENUM('INFO', 'WARN', 'ERROR')	NO	Severity level of the log entry
message	TEXT	NO	Log message content
created_at	DATETIME	YES	Timestamp of log creation

2.5 Transaction_Participants

Junction table linking Users to Transactions with a role. Designed by Nshuti Lancelot.

Column	Type	Nullable	Description
participant_id	INT AUTO_INCREMENT	NO	Primary key
transaction_id	INT	NO	FK → Transactions (CASCADE DELETE)
user_id	INT	NO	FK → Users
role	ENUM('SENDER', 'RECEIVER')	NO	Role of the user in the transaction
created_at	DATETIME	YES	Timestamp of record creation

3. Demo Query Screenshots

Cross-table analytics queries demonstrating the database capabilities.

```
mysql> SELECT * FROM system_logs;
+-----+-----+-----+-----+
| log_id | log_level | message                                     | logged_at |
+-----+-----+-----+-----+
| 1      | INFO     | Successfully parsed SMS at index 1 - received transaction | 2026-05-18 09:18:51 |
| 2      | INFO     | Successfully parsed SMS at index 2 - payment transaction | 2026-05-18 09:18:51 |
| 3      | WARNING  | SMS at index 45 matched no known pattern - skipped      | 2026-05-18 09:18:51 |
| 4      | ERROR    | SMS at index 67 - malformed date field, could not parse | 2026-05-18 09:18:51 |
| 5      | INFO     | Pipeline completed - 1688 records processed, 3 skipped   | 2026-05-18 09:18:51 |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM system_logs WHERE log_level IN ('ERROR', 'WARNING');
+-----+-----+-----+-----+
| log_id | log_level | message                                     | logged_at |
+-----+-----+-----+-----+
| 3      | WARNING  | SMS at index 45 matched no known pattern - skipped      | 2026-05-18 09:18:51 |
| 4      | ERROR    | SMS at index 67 - malformed date field, could not parse | 2026-05-18 09:18:51 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
+-----+-----+-----+-----+
| category_code | description                                     | transaction_count | total_amount_rwf |
+-----+-----+-----+-----+
| received      | money received from another momo user           | 1                 | 2000.00           |
| payment       | outgoing payment to another person              | 1                 | 1000.00           |
| transfer      | money transfer to a phone number                | 1                 | 10000.00          |
| bank_deposit  | cash deposited into momo from bank              | 1                 | 40000.00          |
| airtime       | Airtime purchase using MoMo balance             | 1                 | 2000.00           |
| direct_payment | Payment processed by a merchant or business    | 1                 | 600.00            |
| withdrawal    | Cash withdrawn from MoMo via an agent           | 0                 | NULL              |
| bank_transfer | Transfer from MoMo to a bank account            | 0                 | NULL              |
+-----+-----+-----+-----+
8 rows in set (0.01 sec)
```

4. Validation Rule Screenshots

Validation rules and constraint screenshots provided by Teta Dianah, confirming data integrity across the schema.

```
mysql> ALTER TABLE system_logs
    -> ADD CONSTRAINT chk_log_message_not_empty CHECK (LENGTH(message) > 0);
Query OK, 5 rows affected (0.10 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO system_logs (log_level, message) VALUES ('INFO', '');
ERROR 3819 (HY000): Check constraint 'chk_log_message_not_empty' is violated.
mysql> █
```

log_id	log_level	message	logged_at
1	INFO	Successfully parsed SMS at index 1 - received transaction	2026-05-17 23:56:36
2	INFO	Successfully parsed SMS at index 2 - payment transaction	2026-05-17 23:56:36
3	WARNING	SMS at index 45 matched no known pattern - skipped	2026-05-17 23:56:36
4	ERROR	SMS at index 67 - malformed date field, could not parse	2026-05-17 23:56:36
5	INFO	Pipeline completed - 1688 records processed, 3 skipped	2026-05-17 23:56:36
6	WARNING	Test log entry for CRUD verification	2026-05-17 23:56:36

6 rows in set (0.00 sec)

log_id	log_level	message	logged_at
1	INFO	Successfully parsed SMS at index 1 - received transaction	2026-05-17 23:56:36
2	INFO	Successfully parsed SMS at index 2 - payment transaction	2026-05-17 23:56:36
3	WARNING	SMS at index 45 matched no known pattern - skipped	2026-05-17 23:56:36
4	ERROR	SMS at index 67 - malformed date field, could not parse	2026-05-17 23:56:36
5	INFO	Pipeline completed - 1688 records processed, 3 skipped	2026-05-17 23:56:36

5 rows in set (0.00 sec)

log_id	log_level	message	logged_at
3	WARNING	SMS at index 45 matched no known pattern - skipped	2026-05-17 23:56:36
6	WARNING	Updated test log entry	2026-05-17 23:56:36

2 rows in set (0.00 sec)

```
mysql> SELECT * FROM system_logs;
+-----+-----+-----+-----+
| log_id | log_level | message | logged_at |
+-----+-----+-----+-----+
| 1 | INFO | Successfully parsed SMS at index 1 - received transaction | 2026-05-18 09:18:51 |
| 2 | INFO | Successfully parsed SMS at index 2 - payment transaction | 2026-05-18 09:18:51 |
| 3 | WARNING | SMS at index 45 matched no known pattern - skipped | 2026-05-18 09:18:51 |
| 4 | ERROR | SMS at index 67 - malformed date field, could not parse | 2026-05-18 09:18:51 |
| 5 | INFO | Pipeline completed - 1688 records processed, 3 skipped | 2026-05-18 09:18:51 |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM system_logs WHERE log_level IN ('ERROR', 'WARNING');
+-----+-----+-----+-----+
| log_id | log_level | message | logged_at |
+-----+-----+-----+-----+
| 3 | WARNING | SMS at index 45 matched no known pattern - skipped | 2026-05-18 09:18:51 |
| 4 | ERROR | SMS at index 67 - malformed date field, could not parse | 2026-05-18 09:18:51 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

5. JSON Modeling & Mapping Queries

This section demonstrates how the relational schema is serialized into nested JSON for API consumption.

5.1 Complex Transaction JSON

A complete MoMo transaction with all related data nested inside — category, sender, receiver, and log:

```
{
  "transaction_id": 1,
  "momo_tx_id": 76662021700,
  "amount": 2000.00,
  "fee": 0.00,
  "new_balance": 2000.00,
  "direction": "CREDIT",
  "transaction_date": "2024-05-10T16:30:51Z",
  "raw_body": "You have received 2000 RWF from Jane Smith...",
  "category": {
    "category_id": 1,
    "category_code": "INCOMING_MONEY"
  },
  "sender": {
```

```

    "user_id": 42,
    "full_name": "Jane Smith",
    "phone": "250788000013",
    "user_type": "CUSTOMER"
  },
  "receiver": {
    "user_id": 1,
    "full_name": "Account Owner",
    "phone": "250788000001",
    "user_type": "CUSTOMER"
  },
  "log": {
    "log_id": 1,
    "log_level": "INFO",
    "message": "Transaction parsed successfully"
  }
}

```

5.2 SQL-to-JSON Field Mapping

SQL Table	Column	JSON Field
Transactions	transaction_id, amount, fee...	Root level fields
Transaction_Categories	category_id, category_code	category.category_id, category.category_code
Users	user_id, full_name, phone	sender.* or receiver.*
System_Logs	log_id, log_level, message	log.log_id, log.log_level, log.message

6. Integration Test Screenshots

Cross-table CRUD operations run by Rugwiro Derrick to verify that all 5 tables work together correctly.

Database changed

```
mysql> INSERT INTO Users (full_name, phone, user_type)
      -> VALUES ('Test User', '250788000001', 'CUSTOMER');
Query OK, 1 row affected (0.014 sec)
```

```
mysql> INSERT INTO System_Logs (log_level, message)
      -> VALUES ('INFO', 'Test transaction parsed successfully');
Query OK, 1 row affected (0.005 sec)
```

```
mysql> INSERT INTO Transactions (
      -> momo_tx_id, category_id, amount, fee,
      -> new_balance, direction, counterparty_name,
      -> transaction_date, raw_body, log_id,
      -> sms_unix_timestamp
      -> )
      -> VALUES (
      -> 999999999999, 1, 5000.00, 100.00,
      -> 28300.00, 'DEBIT', 'Test User',
      -> '2024-05-11 20:34:47',
      -> '5000 RWF transferred to Test User',
      -> LAST_INSERT_ID(),
      -> UNIX_TIMESTAMP('2024-05-11 20:34:47')
      -> );
Query OK, 1 row affected (0.035 sec)
```

```
mysql> SELECT
      -> t.transaction_id,
      -> t.amount,
      -> t.direction,
      -> c.category_code,
      -> l.log_level
      -> FROM Transactions t
      -> JOIN Transaction_Categories c ON t.category_id = c.category_id
      -> LEFT JOIN System_Logs l ON t.log_id = l.log_id
      -> WHERE t.momo_tx_id = 999999999999;
```

transaction_id	amount	direction	category_code	log_level
8	5000.00	DEBIT	received	INFO

1 row in set (0.010 sec)

```
mysql> UPDATE Transactions
      -> SET amount = 9000.00
      -> WHERE momo_tx_id = 99999999999;
Query OK, 1 row affected (0.011 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> INSERT INTO Transaction_Participants (transaction_id, user_id, role)
      -> VALUES (LAST_INSERT_ID(), 1, 'SENDER');
Query OK, 1 row affected (0.008 sec)
```

```
mysql> DELETE FROM Transactions
      -> WHERE momo_tx_id = 99999999999;
Query OK, 1 row affected (0.007 sec)
```

```
mysql> SELECT * FROM Transaction_Participants
      -> WHERE transaction_id = 1;
```

participant_id	transaction_id	user_id	role	created_at
2	1	1	RECEIVER	2026-05-18 15:50:38
1	1	2	SENDER	2026-05-18 15:50:38

```
2 rows in set (0.011 sec)
```